TP final : l'objet mesure, publie et décide

Semestre 7 - durée : 3 h - binômes - enseignant : Teewende Boris KIEMA

Ce TP remplace les TP2 et TP3. Il reprend votre montage du TP1 et l'amène jusqu'à un objet complet : il mesure, il envoie ses mesures sur le réseau, et il décide seul d'irriguer.

Objectifs. À la fin de la séance, votre objet :

- publie ses mesures sur le broker de la salle, sous votre propre compte (EX3) ;
- ouvre et ferme la vanne tout seul, **même réseau coupé** (EX4) ;
- n'irrigue jamais plus que la durée maximale (EX5) ;
- se met en sécurité si la sonde est débranchée (EX2).

Deux fichiers vous sont distribués. `agrisense_base.ino` contient tout le travail du TP1, prêt à l'emploi. `agrisense_final.ino` contient le programme complet de la séance, avec **six trous** à remplir, repérés par `« TROU n »`. Vous n'écrivez que ces six trous : le reste est fourni.

Matériel par binôme. Le montage du TP1 : sonde ou potentiomètre sur `A0`, LED et résistance 220 Ω entre `D5` et `GND`. Le bouton n'est plus utilisé.

Ce qui vous sera communiqué. Le nom et le mot de passe du Wi-Fi ; l'adresse du broker ; votre compte `gXX` et son mot de passe.

Déroulé. Essayez de respecter au maximum le chrono : si un exercice n'est pas fini, passez au suivant et revenez-y à la fin.

Exercice	Ce que vous obtenez	Durée
0. Repartir sur une base saine	Une mesure propre, affichée	20 min
1. Publier (EX3)	Votre mesure apparaît à l'écran de la salle	60 min
2. Décider (EX4, EX5, EX2)	La vanne s'ouvre et se ferme seule	55 min
3. Démonstrations de l'enseignant	De quoi remplir le rapport de tests	25 min
4. Enregistrer le travail	Le dépôt à jour	15 min

Deux règles de séance. Un binôme qui a terminé un exercice va aider le binôme voisin le plus en retard. Et un binôme bloqué plus de dix minutes sur le réseau passe directement à l'exercice 2, qui fonctionne sans réseau.

Exercice 0 : repartir sur une base saine

20 min

1. Ouvrez `agrisense_base.ino`. Remplacez `SOL_SEC` et `SOL_HUMIDE` par **vos deux relevés** du TP1. Si vous les avez perdus, refaites-les en deux minutes : sonde à l'air libre, puis sonde dans l'eau.
2. Téléversez. Le moniteur série doit afficher une ligne par seconde, de la forme `sol=47.3 q=0`.
3. À quoi sert `q` ? Débranchez le fil de la sonde : que devient la valeur affichée ? Rebranchez.
4. Mouillez la sonde, ou tournez le potentiomètre : la valeur de `sol` doit varier entre 0 et 100.

Exercice 1 : publier

60 min

Ouvrez maintenant `agrisense_final.ino`. Reportez-y vos deux valeurs de calibration.

1.1 Les secrets (10 min). Créez à côté du programme un fichier `secrets.h` contenant quatre lignes :

```
#define WIFI_SSID "nom-du-reseau"
#define WIFI_PASS "mot-de-passe-du-reseau"
#define MQTT_USER "gXX"
#define MQTT_PASS "votre-mot-de-passe"
```

Pourquoi ces quatre lignes ne sont-elles pas dans le programme principal ? Que se passerait-il si ce fichier partait dans le dépôt Git ?

1.2 Le TROU 1. Remplacez `gXX` par le numéro de votre groupe, aux trois endroits indiqués.

1.3 Le TROU 2. Avant de se connecter, l'objet doit donner son nom d'utilisateur et son mot de passe au broker. Cela se fait en une seule ligne :

```
mqtt.setUsernamePassword(MQTT_USER, MQTT_PASS);
```

Téléversez. Le moniteur série doit afficher l'adresse IP, le signal, puis `MQTT : connecte`. S'il affiche `MQTT refuse`, notez le code affiché et faites des recherches puis appelez l'enseignant.

1.4 Le TROU 3 (20 min). Composez le message publié. Il doit être exactement :

```
{"sol":47.3,"q":0,"etat":"REPOS"}
```

On l'écrit morceau par morceau, comme sur le moniteur série. Le premier morceau vous est donné :

```
mqtt.print("{\"sol\":"); mqtt.print(sol, 1);
// ... puis "}
```

1.5 Vérifier (15 min). Dans un terminal du poste, abonnez-vous à votre branche. Vos messages doivent défiler, et apparaître sur l'écran projeté.

```
mosquitto_sub -h ADRESSE -u gXX -P MOTDEPASSE -t "agrisense/gXX/#" -v
```

Questions, à noter sur votre feuille :

1. Abonnez-vous à la branche du binôme voisin (`agrisense/gYY/#`). Que recevez-vous ? Pourquoi ?
2. Dans `loop()`, la ligne `mqtt.poll();` semble inutile. Mettez-la en commentaire et attendez deux minutes. Que se passe-t-il ? Remettez-la.

Exercice 2 : décider

55 min

Cet exercice ne demande pas le réseau : faites-le même si l'exercice 1 n'est pas terminé.

2.1 Le seuil unique, et pourquoi il ne marche pas (10 min). Ajoutez temporairement, à la fin de `loop()` :

```
digitalWrite(LED_VANNE, sol < 40.0 ? HIGH : LOW); // a retirer ensuite
```

Amenez lentement la mesure autour de 40 %. Qu'observez-vous ? Qu'arriverait-il à une pompe ou à un relais ? Retirez ensuite cette ligne.

2.2 L'automate (35 min). Remplissez les trous 4, 5 et 6 de la fonction `decider()`. Chacun fait une ou deux lignes. Les outils :

```
changerEtat(IRRIGATION);  // pour passer dans un etat
sol < SEUIL_BAS           // le sol est plus sec que le seuil bas
sol > SEUIL_HAUT          // le sol est plus humide que le seuil haut
duree >= DUREE_MAX        // on irrigue depuis trop longtemps
duree >= DUREE_PAUSE      // la pause est terminee
```

2.3 Vérifier (10 min). Sonde à l'air libre, donc sol sec :

1. La LED s'allume, puis s'éteint seule au bout de 10 s. Chronométrez.
2. Elle se rallume 20 s plus tard. L'état affiché suit : `REPOS`, `IRRIGATION`, `PAUSE`, `REPOS`...
3. Mouillez la sonde pendant une irrigation : la LED s'éteint avant les 10 s et l'état revient à `REPOS`.
4. Débranchez le fil de la sonde : l'état passe à `DEFAUT` et la LED s'éteint.

Si l'exercice 1 est terminé, le champ `etat` apparaît maintenant dans vos messages publiés.

Exercice 3 : démonstrations de l'enseignant

25 min

Vous n'avez rien à programmer. Gardez votre objet en marche et **notez ce que vous observez**.

1. **Le réseau coupé (EX4).** L'enseignant arrête le broker deux minutes. Votre objet continue-t-il de mesurer ? Continue-t-il d'irriguer ? Que dit le moniteur série ? Au retour du broker, en combien de temps revient-il seul ?
2. **Ce que voit le réseau (EX7).** L'enseignant projette la capture du trafic. Retrouvez-y votre nom d'utilisateur, votre mot de passe et vos mesures. Qui d'autre peut les voir ? Quelle exigence est violée ? Que faudrait-il changer (CM3) ?
3. **Le tableau de bord (EX3).** L'enseignant montre les mesures de toute la salle sur un tableau de bord Node-RED. D'où vient l'heure de la courbe, puisque la carte ne l'envoie pas ? Quelle erreur cela introduit-il après une coupure de réseau ?
4. **Pour aller plus loin.** Vous pouvez faire des recherche sur le chien de garde, la persistance en mémoire flash et le sommeil profond ; et voir comment on les implémente. Notez à quoi sert chacun, et l'exigence qu'il couvre.

Exercice 4 : enregistrer le travail

15 min

1. Vérifiez que `secrets.h` ne partira pas dans le dépôt.

```
git status -uall
```

2. Complétez `README.md` : vos deux valeurs de calibration, le tableau de vos sujets MQTT, et le nom de votre compte...
3. Enregistrez la séance.

```
git add .
git commit -m "TP final : publication MQTT, automate de decision"
```

4. *Bonus, si tout fonctionne.* Ajoutez le testament : à la connexion, l'objet confie au broker un message « hors ligne » à publier s'il disparaît sans prévenir. Puis débranchez la carte et observez votre abonné.

```
const char HORS_LIGNE[] = "hors ligne";
// dans connecter(), AVANT mqtt.connect :
mqtt.beginWill(SUJET_ETAT, strlen(HORS_LIGNE), true, 1);
mqtt.print(HORS_LIGNE);
mqtt.endWill();
```